

Load Balancing

1. First create two instances
 1. And make sure that allow http is turned on while creating an instance.
2. Install webserver(apache) in both.

```
sudo apt update -y
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
sudo systemctl status apache2
sudo ufw allow 'Apache'
echo "This is Server 1" | sudo tee /var/www/html/index.html
echo "This is Server 2" | sudo tee /var/www/html/index.html
```
3. Then open browser and type http:// . Do for both server 1 and server2.
4. Now create a new target group
 1. Go to **EC2 → Target Groups → Create target group**
 2. Target type: **Instances**
 3. Protocol: **HTTP**
 4. Port: **80**
 5. VPC: same as EC2 instances
 6. Health check path: /
 7. Click **Create**
 8. Now register instances:
 1. Open the created target group
 2. Go to **Targets tab**
 3. Click **Register targets**
 4. Select both instances
 5. Click **Include as pending**
 6. Click **Register pending targets**
5. Create Application Load Balancer.
 1. Go to **EC2 → Load Balancers → Create**
 2. Choose **Application Load Balancer**
 3. Configure:
 4. Scheme: **Internet-facing**
 5. Listener: **HTTP : 80**
 6. Availability Zones: select at least **2 AZs**
 7. Security group: click **Create new security group**
 1. (Inbound) HTTP 80 → Anywhere IPv4

2. Create security group.
8. Now switch to creating application load balancer page
9. Listener routing:
 1. **Forward to → your Target Group**
10. Click **Create Load Balancer**

Wait until:

1. State = **Active**
2. Targets = **Healthy**

6. **Allow traffic only from Load Balancer (Important — do after healthy)**
 1. Now modify the **EC2 instances security group**:
 2. Remove rule: (inbound) HTTP → Anywhere
 3. Add rule:
 4. Type: HTTP
 5. Source: **Load Balancer Security Group**
 6. Do this for both instances.
 7. Now instances accept traffic only from ALB.
 8. Now go to load balancer page.

7. Test Load Balancing

- Go to EC2 → Load Balancers
- Select your load balancer
- Copy DNS name
- Paste in browser

Refresh repeatedly.

You should see:

Server 1 → Server 2 → Server 1 → Server 2

Auto Scaling

1. Create Launch Template

1. Go to **EC2 → Launch Templates → Create launch template**
 2. Give template name
 3. Application and OS Images -> Quick start → **Ubuntu**
 4. Instance type → **t3.micro**
 5. Key pair → select the same key used for EC2 instances
 6. Security group → select the **same security group used by your EC2 instances** (the one allowing HTTP) (two in total)
 7. Go to **Advanced details → User data** and paste:

```
#!/bin/bash
apt update -y
apt install apache2 -y
systemctl start apache2
systemctl enable apache2
echo "<h1>Server $(hostname)</h1>" > /var/www/html/index.html
```
 8. Click **Create launch template**
-

2. Create Auto Scaling Group

1. Go to **EC2 → Auto Scaling Groups → Create**
 2. Give group name
 3. Select the launch template created above
 4. Select two availability zones.
 5. Click next.
 6. Load balancing section
 1. Attach to existing load balancer.
 2. Choose from your balancer target groups
 3. Select your load balancer target group.
 7. Select **VPC** (same as EC2 + Load Balancer)
 8. Select **at least two subnets** (same AZs as Load Balancer)
 9. Click **Next**
 10. Health check section
 1. Turn on Elastic Load Balancing health checks
 2. Click next.
-

3. 5. Group Size

1. Set:
2. Desired capacity: **2**
3. Minimum capacity: **2**
4. Maximum capacity: **4**
5. Click **Next**

4. 6. Scaling Policy

1. Choose **Target tracking scaling policy**
2. Metric type: **Average CPU utilization**
3. Target value: **60**
4. Instance warmup: **300 seconds**
5. Click **Next** → **Next**

5. 7. Tags

1. Add:
2. Key: **Name**
3. Value: **web-asg-instance**
4. Click **Next** → **Create Auto Scaling Group**

6. 3. Test Auto Scaling

1. Connect to one running instance:
`sudo apt install stress -y`
`stress --cpu 2 --timeout 300`
2. ⚠️ Note: scaling takes **4–6 minutes** (warmup + metric delay)
3. After a few minutes:
4. A new instance should appear in EC2
5. It will automatically register in Target Group
6. Load Balancer will start sending traffic

7. 4. Auto Replacement Test

1. If you terminate one instance manually:
2. Auto Scaling Group automatically launches a new instance
3. Registers it to Target Group
4. Load Balancer uses it